Code Walkthrough

Every script, in plain language

13 files in `code/`. **Twelve back a number that is in the paper**; the thirteenth (`fed_family_loco.py`) is the disclosure work, cut only for page count.

The nine superseded scripts were removed from this bundle to keep it focused. They are all still in the main repo under `rebuilt_results/` if he wants them. Part 2 says what each one was and where it went.

Part 1 — The scripts in the paper

1. `rebuild_with_session.py` — builds the data

Plain version: chops the raw city recordings into 60-second windows and works out 30 numbers for each one: average signal strength, how fast you were going, how many times you switched cell tower, and so on.

Why it exists: the old cached windows had no record of *which trip* each window came from. Without that you cannot stop two windows from the same trip landing on both sides of the train/test split. This version adds a `session` column so you can.

The point to make: it imports `window_features()` from the *original* code unchanged. The features are not reimplemented, so they cannot have drifted. The log proves it: same 7,094 windows, same values.

Output: `results/rebuild_session_log.txt` — 228 journeys, four cities.

2. `ladder_grouped.py` — what each group of features is worth

Plain version: trains a random forest four times, each time with fewer features, and reports what you lose. Also runs each under three different train/test splits.

Why it matters: this is the script that answers Khalil's GPS question. It shows exactly what the model scores with and without coordinates and velocity.

Policy	Features	8 modes	4 families
A — all features	30	0.636	0.748

Policy	Features	8 modes	4 families
B — no coordinates	28	0.587	0.694
C — radio only	25	0.479	0.684
D — radio, no campaign fields	16	0.427	0.639

Output: `results/ladder_grouped_rf.csv`, plus `ladder_grouped_fed.csv` for the federated version. Feeds Fig. 1 and the 0.639-against-0.748 sentence in Sec. IV-B.

3. `verify_duplicates.py` — proves the duplicate-ride problem

Plain version: takes the 17 files Khalil flagged as "second phone on a ride we already have" and checks each one two completely separate ways. Then it measures how often those twins end up on opposite sides of the split.

The two checks:

- **Feature distance.** Two phones a few metres apart for a whole ride produce nearly identical window statistics. So compare each flagged file against every other file of the same class in the same city, and find its closest match.
- **Clock time.** Two logs recorded at the same moment cover the same wall-clock interval.

Why two: the clock check alone over-detects, because the logs record time of day with no date, so two rides on different days at the same hour look simultaneous. Any file whose own span runs longer than six hours is dropped as a midnight-wrap artifact, and a pair only counts when *both* checks agree. That leaves **15 of 17**.

```
unique ride pairs: 15, covering 1728 windows (24.4%)
mean 5.5 of 15 pairs separated per seed (37%)
at least one leak in 10 of 10 seeds
```

Run this one live. It takes under a minute.

Outputs: `results/confirmed_pairs.json`, `results/file_timespans.csv`

4. `causal_ridegrouped.py` — the headline experiment (Fig. 3)

Plain version: the paper's main claim is that a class being stuck at *one* site hurts far more than simply having *fewer examples* of it. This script tests it by running three versions of the same experiment:

- **(A)** the data as it is

- **(B)** take a class and delete it everywhere except its biggest holder, so it lives at one site only
- **(C)** delete the *same number* of examples at random, but leave them spread across sites

B and C train on an identical number of samples. The only difference is whether the class sits in one place. So **(B) minus (C)** is the effect of concentration with the count held fixed.

Class	(A)	(B)	(C)	(C)−(A)	(B)−(C)
walk	0.858	0.707	0.847	−0.011	−0.140
car	0.630	0.289	0.577	−0.052	−0.288
bus	0.664	0.162	0.661	−0.004	−0.499

Losing samples costs almost nothing. Concentrating them costs a lot. Negative in **9 of 9** runs.

Nice detail to mention: `scooter` only ever existed at one site, so B and C do nothing to it. Its change is exactly zero in every seed, a sanity check that the machinery isn't inventing differences.

Output: `results/causal_ridegrouped.csv` → Fig. 3

5. `fedobserve_ridegrouped.py` — the main table (Table II)

Plain version: trains four configurations ten times each and compares them on the same splits with the same starting weights, so the comparison is exact.

Configuration	train F1	Macro F1
FedAvg + mask	0.0259	0.6061
Rule + mask	0.1289	0.6187
Paired test	9/10, p = 0.0020	9/10, p = 0.0322

What "the rule" does, in four places:

1. **Loss weighting:** during training, rare classes count for more.
2. **Restoration:** after training, if a site never saw a class, its opinion about that class is discarded and replaced with the shared model's.

3. **Aggregation:** when combining models, only sites that actually saw a class get a vote on it.
4. **Masking:** at prediction time, a site cannot predict a class it has never seen.

Output: `results/fedobserve_ridegrouped.csv`

6. `baselines_ridegrouped.py` — the comparison with prior methods (Table II)

Plain version: runs two published methods, FedRS and FedProx, on exactly the same splits and seeds, so the comparison is fair.

Method	train F1	Macro F1	train p	Macro p
FedProx + mask	0.0254	0.6035	0.0020	0.0322
FedRS + mask	0.1089	0.6173	0.1191	0.4229

This is the uncomfortable one. FedRS now ties on macro F1. See Part 3.

Output: `results/baselines_ridegrouped.csv`

7. `factorial.py` — which part of the rule actually works

Plain version: the rule has three training-time parts. Instead of adding them one at a time, which cannot tell you what any single one does, this runs all eight on/off combinations.

What it found: almost all the gain comes from the **loss weighting**, which is a completely standard technique. The clever-sounding part, the class-wise aggregation weight that prior work focuses on, adds essentially nothing on macro F1.

Why that's fine: the paper says this itself. It never claims the method is novel, and it names cwFedAvg as the source of the aggregation weight.

Output: `results/factorial.csv`

8. `local_bias.py` — the FedPer comparison

Plain version: FedPer is a method where each site keeps its own final layer. It scores higher per site than the paper's approach. This script asks whether that advantage can be recovered more cheaply, by letting each site keep just 8 numbers (an output bias) instead of a whole layer.

Answer: no. The bias trick recovers none of it (2/5 seeds, p = 0.41). FedPer's advantage comes from the local layer genuinely adapting to each site's data, not from anything simpler.

Where it is in the paper: the "+0.049 macro F1 per client" sentence in Sec. VI.

Output: `results/local_bias.csv`

9. `make_figures.py` — draws all four figures

Plain version: reads the saved CSVs and the window cache and draws the four PDFs. Nothing is typed in by hand; every number in every figure comes from a file.

Watch out: the file names do not match the figure numbers.

Printed as	File
Fig. 1	`figM2_protocols.pdf`
Fig. 2	`figM3_campaign.pdf`
Fig. 3	**`figM1_isolation.pdf`**
Fig. 4	`figM4_handover.pdf`

Part 1b — The thirteenth file

`fed_family_loco.py` — the disclosure result, cut for length

Plain version: tests whether *hiding* the GPS coordinates helps the model work in a city it has never seen. It does: +0.047 macro F1 under FedAvg and +0.059 under the rule, over ten seeds.

Why that's interesting: it runs backwards to the usual privacy-versus-accuracy trade-off. Withholding information made transfer better, because the model had been leaning on *where* modes happen rather than what they look like.

Why it's not in the paper: the whole section was cut to reach 6 pages. The effect is also concentrated in one city (KZ, 0.20 to 0.39), and with one such site it cannot carry a section. It goes straight into the journal version.

Output: `results/fed_family_loco.csv`

Part 2 — What was removed from this bundle

Nine scripts were taken out so the folder shows only live work. All of them are still in the main repo at `rebuilt_results/`. If he asks for any, they are one copy away.

The three superseded originals

Removed	Replaced by	Why
`causal_grouped.py`	`causal_ridegrouped.py`	Grouped by file, so twin recordings could split
`grouped_10seed.py`	`fedobserve_ridegrouped.py`	Same
`baselines_grouped.py`	`baselines_ridegrouped.py`	Same

The change is one line. You don't need the old file to show it:

```
# old — the split unit was the recording file
D['sid'] = D.city + "::" + D.session.astype(str)

# new — the split unit is the ride, so twins stay together
D['file'] = D.session.astype(str).str.replace('.csv', '', regex=False)
D['ride'] = D.file.map(lambda f: rm.get(f, f))     # rm =
confirmed_pairs.json
D['sid']  = D.city + "::" + D.ride
```

Everything else in those scripts (seeds, hyperparameters, arms, scoring) is untouched. That is why the old and new results are directly comparable.

The six others

- **`main_table_10seed.py`**, **`main_table_ridegrouped.py`**: the variant that drops the class-wise aggregation weight. At 5 seeds it looked better; at 10 it wasn't (p = 0.78), so the paper keeps the weight. Kept out because they are a check, not a result.
- **`fedavg_masked_10seed.py`**: the fairness fix. The first main run gave the paper's method the inference mask but not the baseline, which isn't a fair test since the mask is free. This trained FedAvg with the mask so like meets like. Now folded into the main runs.

- **`masked_inference.py`**, **`fedper_proper.py`**, **`local_only.py`**: the FedPer investigation. Worth describing even though the files are gone, because it is your best evidence of rigour.

The FedPer investigation, step by step:

1. `fedper_proper.py` ran FedPer properly, reporting both the pooled and the per-site score, since FedPer has no single model.
2. `masked_inference.py` tested the obvious explanation: FedPer wins because its local layer never considers impossible classes. **It closed only about a quarter of the gap.**
3. `local_bias.py` (still in the bundle) tried the cheapest alternative, 8 bias numbers per site. **It closed none of it.**
4. `local_only.py` asked a different question: would a city do better training alone? It reports two scores, one over all 8 classes and one over only the classes the city holds.

Two concrete alternative explanations were tested and both failed. That is why the paper reports FedPer's advantage as real rather than explaining it away.

Part 3 — What he will probably ask

"FedRS ties you now. So what's the contribution?"

Say it straight: on macro F1 it's a tie (5/10 seeds, p = 0.42), and on the isolated class the lead isn't significant either (p = 0.12).

Then give the reading the paper gives: **FedRS recovering most of what the rule recovers is evidence for the diagnosis.** FedRS works by restricting how far absent classes get pushed down during training. If that gets you most of the way, the problem really is *how absent classes are updated*, which is exactly what Sec. V argues.

The paper's contributions are the measurements, not the method, and it says so.

"You report p = 0.002 on a class with two rides."

Agree. With two `train` rides, each seed tests one of them, so ten seeds vary the split and the starting weights, not the test ride. **The paper now says this itself**, immediately after the headline numbers. Saying it before he does is worth a lot.

"Why are some experiments on one split and some on another?"

Honest answer: the main table, the baselines and the isolation experiment are ride-grouped. The random-forest ladder, the factorial and FedPer are file-grouped and were not re-run.

Why it's defensible: each of those is a comparison *at a fixed split*, so the difference between its arms is unaffected. What carries the residual inflation is the absolute values. The Protocol paragraph states this outright rather than hiding it.

"You found a shortcut and kept it in."

PDSCH separates KAUST's four exclusive modes. The isolation experiment manipulates walk, car and bus, classes that field pattern does **not** separate. So the criticism lands on the method section, not on the headline result.

What is true: Secs. V–VI were not re-run on the 16 radio-only features. That is the first job for the journal version.

"Why don't my numbers match yours?"

They were never comparable:

	His notebook	The paper
Classes	3 (chance 0.33)	8 (chance 0.125)
Windows	120 s, 50% overlap	60 s, no overlap
Clients	3 regions, KZ held out	4 cities, KZ included
Features	33, no GPS/speed/time	30, GPS included
Files	Drops 20	Keeps all

His 0.532 and the paper's 0.60 are different tasks, not a discrepancy.

Hand him this

His section 8, where every method drops to 0.38–0.41 on a region never seen in training, **independently replicates the paper's transfer-collapse finding**, on a different pipeline, with different features and different methods. That is a real corroboration and he should hear you say so.

Part 4 — Gaps, say them first

1. **Mixed splits.** Three experiments ride-grouped, three file-grouped. Disclosed in the Protocol paragraph.
2. **`train` is pseudo-replicated.** Two rides, so the seeds don't give independent test rides. Now stated in the paper.
3. **Campaign fields kept** in the federated experiments. Sec. IV-B reports what removing them costs on the random forest, but Secs. V–VI weren't re-run on the reduced set.
4. **`ablation_final.py:27`** in the main repo pools the scaler across cities while the text claims per-site statistics. Pre-existing, unresolved.
5. **`README.md` in this folder is stale:** dated 29 Oct 11:16, before the ride-grouping work, so it still shows the old numbers. Point him at this file and `WALKTHROUGH.md` instead.
6. **Nine scripts were removed from this bundle** and live in `rebuilt_results/`. Say so up front rather than letting him notice gaps in the numbering.

One line for the start of the meeting

He raised three objections. All three were right. All three are fixed, two of them made the paper stronger, and the third, the duplicate rides, came out of his own cleaning step.